Social Marketing / Social AI

About the product

Social AI is a SaaS product that helps businesses manage their social media presence more efficiently by reducing repetitive and time-consuming tasks. It allows a business to connect all its social media accounts once and publish content across multiple platforms from a single dashboard. It supports Facebook Pages, Instagram, Google Business Profile, LinkedIn, X, TikTok, YouTube, as well as WordPress websites and blogs.

A business can schedule posts in advance using a content calendar or publish them immediately whenever required. It also provides the option to save unfinished posts as drafts and continue editing them later. When the team runs out of content ideas, the built-in AI can generate captions and create images based on a simple prompt. It can also fetch ready-to-use images and videos from stock libraries, recommend relevant hashtags, and reuse previously saved templates.

Once a post is published, the business can track its performance through analytics and generate on-brand AI replies for customer comments. For businesses operating multiple branches or locations, a single post can be published across all branches simultaneously, ensuring consistent communication.

Vendasta offers Social AI as a white-label solution, enabling agencies to resell the product under their own brand and provide social media management services to their local business clients.

Questions that follow from this summary

Once the summary above is said out loud, the natural next questions are about how those capabilities actually work. Short, honest answers follow.

When a business connects an account once, what happens behind that? The business authorises each network through its OAuth flow, and the connection is saved as a record tied to that business, called a social service. The access tokens for most networks are held server-side in the older Core Services system, while Instagram and X keep their own. The user never sees or handles a token, and a reconnection is only needed if a token later expires.

How does publishing across many platforms from one dashboard actually work? One request starts a separate publishing job for each network, and each job runs as its own durable workflow. The networks are handled independently and in parallel, so a slow or failing one does not hold up the rest.

If a post goes to five platforms and one fails, what happens to the other four? Because each network is its own workflow, four can succeed while one fails. The failed one retries within limits, and if it still cannot publish it falls back to a draft for that network, while the four successful posts stay live. It is never all-or-nothing.

How does scheduling stay reliable if a server restarts before the post is due? Scheduling runs on durable Temporal workflows that sleep until the due time and survive restarts and deployments. It is not an in-memory timer that would be lost when a process restarts.

How does the AI keep captions on-brand rather than generic? Before writing, it retrieves context about that specific business and adds it to the prompt, so the words reflect the business's own voice rather than a generic shop. The prompts themselves are kept centrally so they stay consistent across the product.

Which AI models are used, and what about cost and limits? Captions use an OpenAI model, and images use Google's Gemini model on Vertex AI, with an OpenAI image model as a fallback. Usage is metered, and image

generation is rate-limited per business so a single account cannot run up unlimited cost.

How does one post reach every branch of a multi-location business? A multi-location post fans out into one post per branch, and each targets that branch's own connected accounts. The posting service coordinates the fan-out so the message stays consistent everywhere.

What does white-label mean here in practice? The same services serve many partners at once, scoped by partner and by business, and the branding, domain, and access are presented per partner by the platform. So an agency's client sees the agency's brand rather than Vendasta's. The exact branding mechanism is handled at the platform level, which I would describe as platform-provided rather than something inside these services.

Each network has different rules and rate limits. How is that handled? Each network has its own integration, and the network-specific API calls are made by the service that owns them, mostly Core Services. Bounded retries and error classification keep a network's temporary hiccup from turning into a duplicate post or a lost one.

Where do the analytics come from? There are two borders worth drawing clearly. The first is origin v/s ownership. The raw numbers, such as reach, impressions, likes, and comments, come from each social network's own API. The networks are the source of truth for performance, and we do not compute those figures ourselves.

Our side pulls them in and presents them per post, so the product is a collector and presenter of the networks' metrics, not the calculator of them. In the code, social-posts exposes a post-performance service with a stats read and a CSV export, and the per-network services fetch network-specific figures, for example the Instagram service has a business-stats call and the Google Business service ingests its own insights.

The second border is what I can state with confidence. I have confirmed the services and calls that surface analytics, but not the full list of metrics, nor the exact fetch path for every network, meaning whether a given number is pulled through Core Services or through the per-network service directly. I would verify that before quoting specific figures or a precise per-network flow.

Consider Priya, who runs a small bakery on a busy street. She has a Facebook page, an Instagram account, and a Google Business listing, and she knows she should post on all of them regularly. Every week she means to share a photo of the day's fresh bread, announce a weekend offer, or reply to a customer's comment. By evening, after baking and billing and closing the shop, she is too tired to log into three apps and write three posts. So the pages go quiet, and quiet pages lose customers.

This is the everyday problem Social Marketing was built to solve. The product, now renamed Social AI, is Vendasta's way for a business like Priya's to manage all of its social media from one place. She writes a post once, and the product publishes it to every network at the same time. If she does not know what to write, the built-in AI drafts the caption for her and even generates the image. She can schedule a whole week of posts in one sitting, and later she can see how each one performed and reply to the comments it collected.

In short, Social AI takes the parts that make social media tiring, the switching between apps, the blank page, the forgetting, and the guesswork, and quietly handles them so that a busy owner can stay present online without spending her evenings on it.

Who uses it, and how it is sold

To understand the product, it helps to know how it reaches Priya in the first place. Vendasta does not sell to her directly. It builds the platform and sells it to partners, which are marketing agencies and media companies, and those partners resell it to their own local-business clients under their own brand.

So there are three layers of people in the story. There are the partners who pay Vendasta, the local businesses who are the partners' clients, and the everyday users who actually log in, usually the business owner or a marketer at the agency looking after many clients at once.

Inside the code, one business is called an account group, and its identifier travels with every post and every connected account. When a brand has many outlets, say a bakery chain with twenty branches, one post can be written once and sent out to every branch's own accounts together. That is the multi-location feature, and it matters a great deal to franchises.

What it can do

At its heart, Social AI lets a business connect its accounts once and then post everywhere from a single screen. The networks it reaches are Facebook Pages, Instagram, Google Business Profile, LinkedIn for both companies and people, X which used to be Twitter, TikTok, and YouTube, along with WordPress and blogs. A connected account is stored as a record the product calls a social service, and each one carries its type, such as a Facebook page or an Instagram user.

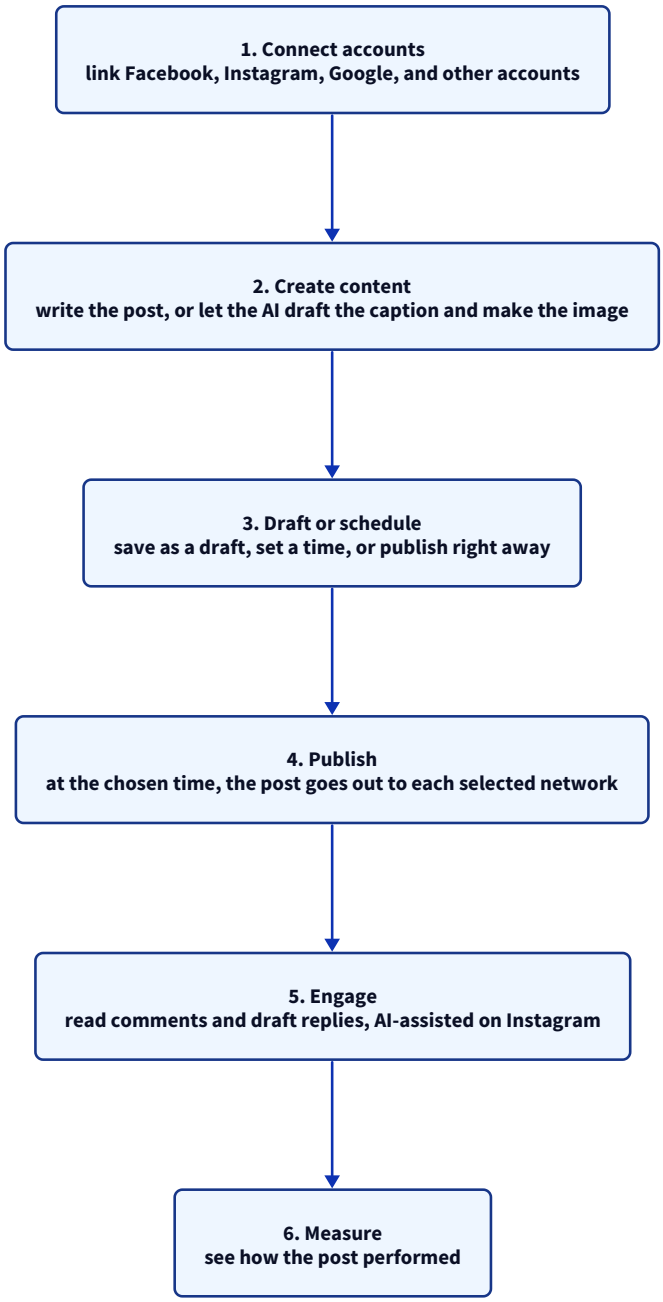
Around that core sit the features that make the product pleasant to use. A business can schedule posts on a calendar or publish them at once. It can save unfinished work as a draft and return to it later.

When inspiration runs dry, the AI writes the caption and generates or edits the image from a short prompt, and if a stock photo is wanted instead, the product pulls images and short clips from libraries like Pexels, Pixabay, Unsplash, and Tenor. It suggests hashtags, keeps reusable templates, and groups posts into campaigns.

After a post goes out, it gathers performance figures and can even help draft on-brand replies to the comments that arrive. There is also a video-generation feature on the horizon, designed and specified in the code but not yet running, so it is best described as something coming rather than something here.

The journey of a single post

The clearest way to see the product is to follow one post from beginning to end.

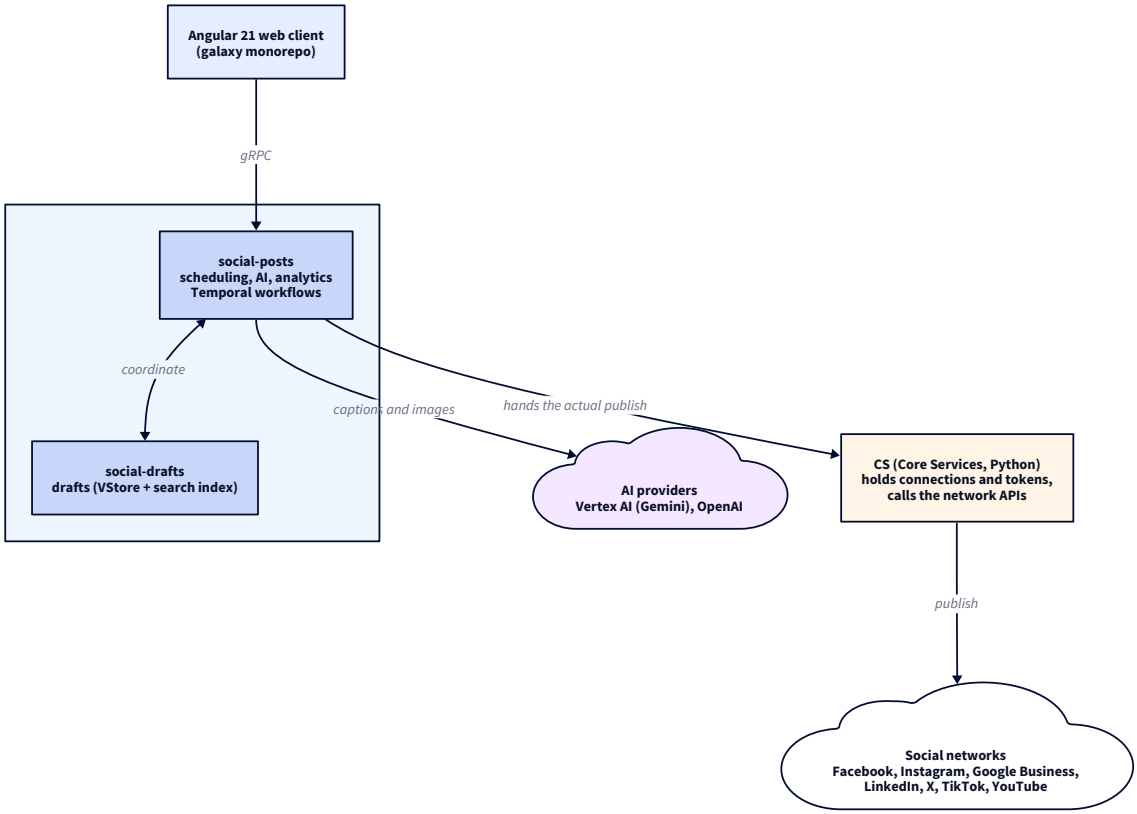


The journey of a single post

Priya begins by connecting her accounts, which she does once and rarely thinks about again. When she wants to announce a Saturday special, she opens the product on Thursday, asks it to write a caption, generates a photo of the special, chooses Facebook and Instagram and her Google listing, and sets it to go out on Friday evening. She then closes the app and forgets about it. On Friday evening the post appears on all three networks on its own. Over the weekend a few customers comment, and on Monday she checks how many people it reached. The quiet, reliable middle of that journey, the scheduling and the publishing, is the part I worked on most closely.

What happens behind the scenes

Behind that simple experience sits a system that is midway through a long and deliberate change. The product began years ago as a single large application written in Python, and over time that monolith is being carefully broken into smaller Go services, with a new web interface built on top. Nothing is thrown away in a hurry. The old and the new run side by side and are kept in step while the work continues.



Social AI: what happens behind the scenes

When Priya schedules her Saturday post, a good deal happens quietly. The web interface she uses is a modern Angular application that lives in the company's shared galaxy codebase, and it speaks to the backend over gRPC using a generated software kit, so the app and the services always agree on the shape of the data. Her request lands in a Go service called social-posts, which is the brain of the operation. It looks after the calendar, coordinates drafts, runs the AI copy and image features, handles the multi-location fan-out, and gathers the statistics.

The part worth pausing on is how the scheduling actually works, because it is not a simple timer. For each post, and for each network the post is going to, social-posts starts a durable workflow using a system called Temporal. That workflow quietly sleeps until the scheduled time arrives. When it wakes, it reads the post once more, so that any edit Priya made in the meantime is picked up, and only then does it publish.

Because the workflow is durable, a post scheduled a fortnight in advance survives server restarts and deployments without being lost, and its retries and timeouts are handled for it. If Priya edits a scheduled post, the running workflow is stopped and a fresh one takes its place. If she deletes it, the workflow simply ends.

For the final step of actually calling each network, social-posts usually does not make the call itself. For Facebook, LinkedIn, X, and Google Business Profile, it hands that step to an older and larger service called CS,

short for Core Services, which holds the account connections and the access tokens and speaks to the networks. Instagram is the exception in the family, because it has its own dedicated service and keeps its own tokens, and can publish on its own.

This reliance on CS is a piece of the older world that the team is slowly unwinding, but for now it is how the posts reach the world.

The services that share the work

If social-posts is the brain, several other services share the load around it. A companion Go service called social-drafts looks after drafts, keeping them in the internal datastore with a search index alongside so they can be found and filtered quickly. The two services work closely, and there is a quiet piece of cleverness between them. If a scheduled post fails to publish, most often because an account's token has expired, it is not simply lost. It is turned back into a draft, so that Priya can reconnect her account and try again, with her words and image intact.

Behind these sit the older parts of the world. The original Python application, called SM, still holds some data and logic and once served the old web pages, and functionality is steadily being moved out of it. CS remains the keeper of connections and the one that publishes to most networks.

Around the edges are the network specialists. The Instagram service handles Instagram's own sign-in, media, comments, and statistics. The Facebook service looks after lead forms, WhatsApp, and Messenger. The Google Business service handles insights, questions and answers, and messaging, acting mostly as a fast local copy of data whose true home is CS.

There is also an AI platform called ai-assistants that holds the prompts and the defined abilities the product's AI features draw upon.

The technology, and why it was chosen

Each choice in the system was made for a reason worth being able to explain.

The new services are written in Go because it is fast, simple, strong at handling many things at once, and it is the company standard. Moving off the single Python application let each piece be deployed on its own and gave better performance on the kind of traffic this product sees, which is a high number of calls carrying small payloads with a lot of fan-out reads.

The services talk to each other and to the web app over gRPC with Protocol Buffers, because that gives a shared, strongly-typed contract, generated client libraries, and lower latency than plain JSON over the older HTTP. For a public website meant for browsers, plain REST would still be the sensible choice, but here the traffic is internal and app-to-service, where gRPC fits well.

Scheduling runs on Temporal because a post may wait for days or weeks before it goes out, and it must survive restarts and crashes in between. Temporal provides that durable execution, along with retries and timeouts, so the team did not have to build a fragile scheduler and a separate table of jobs by hand. Data owned by a service lives in the company's internal datastore, with a search index added for drafts where searching matters. The Go services run on Kubernetes through the platform's tooling, and their health is watched through service-level objectives rather than raw alarms, so that the on-call engineer is woken for real problems and not for noise.

The AI features lean on two providers. Image generation uses Google's Gemini image model through Vertex AI, with an OpenAI image model as a fallback, and caption writing uses an OpenAI model. One point I keep straight

when I describe this: these AI calls go to OpenAI and Vertex AI directly, or through the ai-assistants platform, and not through the inference-gateway. That inference-gateway is a separate side project of mine, and I never let the two get muddled.

Where the intelligence comes from

The rename from Social Marketing to Social AI reflects real, working intelligence rather than a fresh coat of paint.

When Priya asks the product to write her caption, it does not simply throw her request at a general model and hope. First it gathers context about her specific business, drawing on stored knowledge through a retrieval step, and adds that to the instructions before asking the model to write. The result is copy that sounds like her bakery rather than a generic shop. This retrieval-then-generate approach is what keeps the writing grounded.

For the image, she gives a short description and the product generates or edits a picture using the Gemini image model on Vertex AI. When a customer later leaves a comment, the product can read the comments on the post and draft a reply that stays on-brand and true to the business, a feature that today works for Instagram. All of these abilities are defined and their prompts are kept in the central ai-assistants platform, so they are consistent and versioned rather than scattered through the code. A video-generation feature has been designed in the same spirit and given a full specification, but it is not yet running, so it belongs to the roadmap.

My part in the story

I work across both the Go services and the Angular interface, and the part I can speak to most deeply is the pipeline that handles posts and drafts.

I designed and built the Go services on Kubernetes that carry a post from scheduling through to publishing, with careful attention to deadlines and to bounded retries. The guiding idea was simple to state and hard to get right: a scheduled post should either publish exactly once or fail cleanly and fall back to a draft, and it should never be silently lost and never be posted twice.

Getting the failure behaviour right was the real work. When a network’s API misbehaves, and the worst case is one that quietly accepts a post but then reports a timeout, the code recognises that situation and refuses to retry into a duplicate. Through a quarter in which failures from the networks roughly doubled, that discipline is what let us hold our availability target.

I also led the move of this functionality off the old Python application and onto the Go services that speak gRPC. Before choosing, I weighed gRPC against staying on REST for the shape of traffic we actually had, and gRPC won on latency for our many small fan-out calls, which brought the tail latency on the busy path down noticeably.

Alongside that, I rebuilt the build-and-deploy pipeline with multi-stage Docker images and tests that run in parallel, which cut deployment time sharply and removed the maintenance windows we used to schedule. I encouraged test-first development across the services and wrote the testing guide the team adopted, and I moved our alerting onto service-level objectives so that pages meant something again.

Along the way I took on technical-lead work, running design and code reviews across the services and guiding two engineers through their first on-call rotations and their first production incidents.

To be honest about the boundaries, the AI writing and image features were built alongside me by the team. I understand well how they fit together, the grounded caption pipeline, the Gemini image path, and the shared

prompts, and I can walk through the architecture, but I lead with the posting, drafts, migration, and reliability work, because that is mine.

A few real situations

The product is easiest to trust when you see it handle ordinary days.

On a good day, Priya asks for a caption about Friday's fish fry, generates an image, picks three networks, and schedules it for Thursday evening. Three quiet workflows sleep until Thursday and then post to each network, and she never thinks about the machinery.

On a busy day at an agency, a single marketer looks after fifty client businesses, moving between them and scheduling content for each, and using the multi-location feature to push one announcement out to every branch of a franchise at once.

On a bad day, a post fails because a Facebook token has gone stale. Instead of vanishing, the post turns itself back into a draft. Priya reconnects the account, submits again, and nothing is lost and nothing is duplicated. It is on the bad days that the careful design earns its keep.

Senior-level questions on the product and my work

For a senior role, the questions move past what the product does and into how it is designed, where it breaks, why each decision was made, and how I lead the work. These are the ones I prepare for, grouped by theme.

Design and scale

If you designed the scheduling and publishing system from scratch today, how would it look? The core shape would be the same: a durable workflow engine, Temporal, with one workflow per post per network, because the real requirements are long waits, surviving restarts, per-network independence, and handling edits and deletes mid-flight. I would fix the two legacy pieces: make the publish call idempotent at the boundary so a retry is always safe, and let each network's own service perform its publish instead of routing through the older Core Services. I would keep the read model separate from the write model, which the current v1 and v2 split already leans toward.

How would this handle ten times the posts, and where does it break first? Temporal scales by adding workers, so the scheduling layer absorbs load horizontally. The first pressure points are the networks' own rate limits and Core Services as a shared publish hub. I would add per-tenant and per-network rate limiting and queueing so a burst from one partner does not starve the others, and I would remove the shared-hub bottleneck by moving publish into each network's service. After that, the draft search index and the stats queries are the next things I would review.

How do you guarantee exactly-once publishing when a network can time out after it has already accepted the post? Honestly, across a boundary I do not control, the achievable target is effectively-once, not true exactly-once. Today we bound retries and classify the timed-out-but-already-posted errors as not worth retrying, so we do not retry into a duplicate. The proper fix is an idempotency key on each publish so a retry the network has already seen becomes a no-op, and that key belongs at the Core Services boundary. As a backstop, reconcile after the fact by checking whether the post actually landed before ever creating a second.

Reliability and correctness

What happens to a workflow that keeps failing, a poison post? Retries are bounded, a fixed number of attempts with a per-activity timeout, so nothing retries forever. Once the bound is hit, the post falls back to a

draft and surfaces to the user, and the failure shows up in metrics. A genuinely stuck workflow is visible in Temporal and can be terminated. The deliberate choice is to fail loudly to a draft rather than loop quietly.

How do you keep data consistent between the old Python app and the new Go services during the migration? They are bridged with events rather than a shared database. The old app publishes changes on Pub/Sub topics and the Go services subscribe, so a draft changed in one place reaches the other. It is eventual consistency, which is the right call for a gradual migration, and it means the consumers must be idempotent and the design must allow for reconciliation. The end state is a single owner per concept so the bridge can be removed.

How do you observe this in production, and how did you set the SLOs? Metrics on publish success and failure per network, on workflow durations, and on backlog, with alerting tied to service-level objectives rather than raw thresholds. That way we are paged on a user-visible symptom, such as the publish success rate falling below target, not on every transient blip. Moving our alerting onto SLOs is specifically what cut the false pages the on-call rotation used to get.

The gRPC migration

You chose gRPC over REST. What did you measure, and when would you not choose it? I compared latency and payload overhead for our actual traffic, which is a high number of small calls with heavy fan-out reads. gRPC on HTTP/2 with protobuf won on tail latency and gave us generated, type-safe clients that keep the web app and the services in step, and it dropped p99 on the hot path. I would not choose it for a public, browser-facing, third-party API, where REST and JSON are easier to consume and debug; there I would keep REST or put a gateway in front.

How did you migrate off the monolith without a risky big-bang cutover? The strangler pattern. Stand up the Go service, move one capability at a time, bridge the state with events during the overlap, and keep the old path alive until the new one is proven. Feature flags and running both paths in parallel let us shift traffic gradually and roll back a single capability without disturbing the rest.

Quality and testing

How do you test a Temporal workflow that sleeps for days and calls external systems? Temporal's test framework runs a workflow with time skipped forward, so a two-week sleep completes instantly, and the activities are mocked so no real network call happens. That makes the scheduling logic, the edit-and-restart path, the delete path, and the retry and fallback behaviour deterministic to test. The activities that call external systems get their own unit tests with the client mocked, plus a small set of opt-in integration tests against the real API that are run by hand, never in the normal pipeline.

You drove test-first development across the services. How do you get a team to actually adopt it? Make it the easy path rather than a lecture. I wrote the testing guide and set up the harness so adding a test is simple, made coverage visible in the pipeline, and reviewed for tests in every pull request until it became the norm. The argument that won the skeptics was the result: production bug volume from those services came down.

Leadership and judgment

Tell me about a production incident you handled. The one that mattered was a quarter when failures from the social networks roughly doubled. Because failed publishes fell back to drafts and retries were bounded and duplicate-safe, users saw posts they could resubmit rather than lost or duplicated content, and we held the availability target. My part was the error-classification and fallback design that turned a bad quarter of provider flakiness into a survivable one, and guiding two engineers through their first incidents on that system.

How do you run a design review, and what do you push on? I push on failure modes and ownership first: what happens when this call fails, times out, or is retried, and who owns the data. For this product that means

asking about idempotency, deadlines, and the edit-or-delete-mid-flight cases before anyone talks through the happy path. I would rather spend the review on the three ways a design breaks than on the one way it works.

The dependency on Core Services for publishing is a known wart. Why is it still there, and how would you remove it? It exists because Core Services already owns the connections, the tokens, and the working network integrations, so delegating to it was the safe and fast path during the migration. Removing it is real work: move token ownership and the network API calls into each network's own service, add an idempotent publish at that boundary, and cut over one network at a time behind flags. It is sequenced after the higher-value migration work, which is a deliberate choice about priorities, not an oversight.

How would you add a new social network to the product? Model its connection as a new social-service type, add its OAuth and token handling, and implement a publish activity inside a network workflow that mirrors the existing ones: sleep until the time, re-read the post, publish, classify the errors, and fall back to a draft on failure. Then surface it in the web client through the generated SDK. The workflow scaffold is shared, so most of the effort is the network's own API quirks and its rate limits.

How would you keep AI cost under control as usage grows? Rate-limit per business, which image generation already does, cache where prompts repeat, choose the smaller model when it is good enough, and track cost per generation as a first-class metric beside latency. The grounding step helps too, because a stronger first draft means fewer regenerations.

Explaining it in two minutes

I work on Social AI, which was called Social Marketing until recently. It lets a local business manage all of its social media from one place. Instead of logging into Facebook, Instagram, Google, LinkedIn, and X separately, the business writes a post once, the AI can draft the caption and make the image, and the product schedules it and publishes it to every connected network at the same time. It also tracks how each post did and helps reply to comments.

Vendasta sells this to partner agencies, who resell it to their local-business clients under their own brand, so it is a white-label product inside a bigger platform.

Underneath, the product is moving from an older Python application to Go services that talk over gRPC, with a modern Angular interface. Scheduling runs on Temporal, so a post set for next week reliably goes out even across restarts and deployments.

My part is the backend pipeline for posts and drafts, the scheduling and publishing, and making it reliable when a network fails, so a post either publishes cleanly or drops back to a draft, never lost and never duplicated. I also led the move of these services from REST to gRPC and rebuilt the deployment pipeline.

Explaining it in five minutes

Social AI is Vendasta's product for managing social media. The problem is simple. A local business, say a dentist or a bakery, has accounts on several networks and no time to post to each one by hand, so the pages go quiet. Social AI lets the business write and schedule a post once and publish it everywhere, with the AI helping write the words and make the image, and with figures afterwards that show what worked.

The way it is sold shapes the product. Vendasta sells the platform to partners, which are agencies and media companies, and they resell it under their own brand to local businesses. In the code, one business is an account group, and a chain with many branches can write one post and send it to every branch at once, which is the multi-location feature.

The main abilities are posting to Facebook, Instagram, Google Business Profile, LinkedIn, X, TikTok, and YouTube, scheduling on a calendar, saving drafts, generating captions and images with AI, drafting replies to comments, and reporting on performance, along with templates and campaigns to keep things organised.

Underneath, the product is midway through a change. It began as a single Python application, and that is being broken into Go services with a new Angular interface. The two services I work on most are social-posts, which is the scheduling and coordinating brain, and social-drafts, which looks after drafts. The interface is a modern Angular app that talks to the backend over gRPC.

The most interesting part is scheduling and publishing. Scheduling runs on Temporal. Each post, for each network, becomes a durable workflow that sleeps until the chosen time, then reads the post again to catch any edits, then publishes. Temporal gives durability and retries for free, which matters because posts can be scheduled far in advance and must survive restarts.

For the actual network call, social-posts hands the work to an older service called CS, which holds the connections and tokens, except for Instagram, which has its own service.

Reliability is the theme I care about most. Retries are bounded, and errors that would cause a duplicate are recognised and not retried. If a post finally fails, usually because a token expired, it turns back into a draft so the owner can reconnect and try again.

My role spans the stack but leans to the backend. I own the posts-and-drafts pipeline, I led the move from REST to gRPC, I rebuilt the deployment pipeline with multi-stage Docker and parallel tests, and I moved our alerting onto service-level objectives. I also mentor two engineers and run design and code reviews.

Explaining it in ten minutes

Let me tell it from the beginning, then go into how it works, then my part, and finally what I would change.

Social AI, once called Social Marketing, is Vendasta’s product for managing social media. Vendasta sells to partners, which are agencies and media companies, and those partners resell to local businesses such as restaurants, dentists, and shops, all under their own brand. So the product is white-label and lives inside a much larger platform rather than standing alone. In the code, a single business or branch is an account group, and that identifier runs through everything.

The problem is that a local business has a presence on many networks and neither the time nor the skill to keep them all active and consistent. Social AI answers this in three ways. It publishes to every network from one place, so the owner writes once and posts everywhere. It uses AI to solve the blank-page problem, drafting the caption and generating an image so the owner edits rather than starts cold. And it shows what worked, with a calendar to keep posting regular and figures to prove the value.

The abilities, in plain terms, are publishing to Facebook, Instagram, Google Business Profile, LinkedIn, X, TikTok, and YouTube, along with WordPress and blogs, scheduling with a calendar, saving drafts, fanning one post out across many branches for franchises, generating captions and images with AI, drafting replies to comments, pulling stock media, suggesting hashtags, keeping templates and campaigns, and reporting on performance.

Now to how it works, which is the interesting part because it is a live migration. The product started as a single Python application called SM, with its own old web pages. That is being broken into Go services with a new interface, and the change is made carefully, keeping the old and the new in step.

The interface today is a modern Angular application in the company’s shared galaxy codebase, and it speaks to the backend over gRPC using a generated kit, so the two always agree on the data.

On the backend, the service I work on most is social-posts, written in Go. It is the brain: it looks after the calendar, coordinates drafts, runs the AI writing and image features, handles multi-location fan-out, and gathers statistics. Beside it is social-drafts, another Go service that looks after drafts, stored in the internal datastore with a search index for quick finding.

An important point is that social-posts usually does not call the networks itself. For Facebook, LinkedIn, X, and Google Business Profile, it hands that step to an older service called CS, which holds the account connections and tokens and speaks to the networks.

Instagram is the exception, with its own service and its own tokens. This dependence on CS is a piece of the old world that the team is slowly unwinding.

Scheduling deserves the most attention. It runs on Temporal rather than a cron job or a plain queue. When a post is scheduled, social-posts starts one durable workflow for each post and each network. The workflow sleeps until the chosen time, then reads the post again so it picks up any edit made while it waited, then publishes.

Temporal matters for three reasons. A post scheduled a fortnight ahead must survive restarts and deployments, and a durable workflow does. The retries and the timeout are built in. And editing a scheduled post cleanly stops the old workflow and starts a new one, while deleting simply ends it. Doing all of that with cron and a queue would mean building durability and many edge cases by hand.

Reliability under failure is the theme I care about most. The hardest case is a network that times out after it has already accepted the post. If we blindly retried, we would create a duplicate, so the publish path recognises those errors and does not retry them. The proper long-term cure is to make the publish call safe to repeat, which lives in CS, and I am honest that today it is handled by recognising the error rather than fully solved.

The other half is that a post which genuinely fails, usually because a token expired, is turned back into a draft, so the owner reconnects and resubmits. The principle is to publish cleanly or fail cleanly, never lose and never duplicate. Through a quarter when failures from the networks roughly doubled, that handling is what held our availability target.

On the AI side, which is why the product was renamed, caption writing uses an OpenAI model with a retrieval step that pulls the specific business's context into the prompt before writing, so the copy is grounded in that business rather than generic. Image generation uses Google's Gemini image model on Vertex AI, with an OpenAI image model as a fallback. Comment replies read a post's comments and draft on-brand responses, working today for Instagram.

The prompts and abilities are kept centrally in the ai-assistants platform rather than hardcoded. A video feature using Google's Veo has been designed and specified but is not yet running, so I call it roadmap. One point I keep straight: these AI calls go to OpenAI and Vertex AI directly, or through the ai-assistants platform, and not through the inference-gateway, which is a separate side project of mine.

My role spans the stack but leans to the Go backend. The pipeline I own is posts and drafts, the Temporal-based scheduling and publishing, the bounded retries and deadlines, and the draft fallback on failure. I led the move of these services off the Python application and onto gRPC, weighing the protocols against our real traffic, which is many small fan-out calls, before choosing gRPC, which brought the tail latency down.

I rebuilt the deployment pipeline with multi-stage Docker and parallel tests, which cut deployment time and removed maintenance windows. I encouraged test-first development and wrote the testing guide the team uses, and I moved alerting onto service-level objectives to cut false alarms. I also do frontend work on the Angular interface, run design and code reviews, and mentor two engineers through on-call and incidents.

If asked what I would improve, I would finish freeing the networks from CS so each is owned by its own service with a publish call safe to repeat, and I would complete the move off the old Python application so that each idea has a single home instead of the bridged state of today. Both turn a working migration into a clean one.

The short version

Social AI, once Social Marketing, is Vendasta's product for managing social media from one place. Agencies resell it to local businesses, one business being an account group. A business posts once and reaches Facebook, Instagram, Google Business Profile, LinkedIn, X, TikTok, and YouTube. The interface is a modern Angular app; the backend is Go services, chiefly social-posts for scheduling and social-drafts for drafts, moving off an older Python application. Scheduling runs on Temporal, one durable workflow per post per network. An older service called CS holds the connections and publishes to most networks, while Instagram runs its own. Reliability comes from bounded retries, recognising duplicate-causing errors, and turning a failed post back into a draft. The AI writes grounded captions, generates images with Gemini on Vertex AI, and drafts Instagram comment replies, with Veo video still on the roadmap. My part is the posts-and-drafts pipeline, the move from REST to gRPC, the deployment pipeline, test-first practice, service-level alerting, frontend work, and technical-lead duties.